

HazardOS — QA Tester Guide

*For Roy Tolosa and Sophie Hope. Read this before you start testing — it's the context and setup you need. Your step-by-step checklist lives in the companion **HazardOS QA Tracker** spreadsheet.*

1. What HazardOS is

HazardOS is a **mobile-first SaaS platform for environmental remediation companies** — the small-to-mid-size businesses that do asbestos, mold, lead paint, and vermiculite abatement. It runs the whole business cycle in one place: lead capture, CRM, mobile site surveys, estimates, proposals with e-signature, job scheduling and crew assignment, time/materials tracking, invoicing, payments, customer feedback, and analytics.

Positioning: **"the operating system for environmental remediation companies."** Tone should read **professional but not stuffy, technical but not complicated, serious about safety but modern about delivery**. It's a B2B tool — if something reads like a consumer app, flag it.

2. Your test accounts (Acme Remediation)

Each tester has a full set of role accounts pre-provisioned in the **Acme Remediation** test org, so you can jump straight into role testing. **Each tester drives their own pool** — so you two can test at the same time without colliding on sessions or data. Every alias lands in that tester's real inbox, but the app treats each as a separate account, so one person can also be signed in as several roles across different browser windows.

Role	Roy (field tester)	Sophie (back-office tester)
Tenant Owner	roy.tolosa+owner@asymmetri c.pro	sophie.hope+owner@asymm etric.pro
Admin	roy.tolosa+admin@asymmetri c.pro	sophie.hope+admin@asymm etric.pro
Estimator	roy.tolosa+estimator@asymm etric.pro	sophie.hope+estimator@asy mmetric.pro
Technician	roy.tolosa+technician@asym metric.pro	sophie.hope+technician@asy mmetric.pro
Viewer (read-only)	roy.tolosa+viewer@asymmetri c.pro	sophie.hope+viewer@asymm etric.pro

- **Shared password: HazardOS-QA-2026!** — emails are pre-confirmed, so sign in immediately (no confirmation step).
- Roy's base account `roy.tolosa@asymmetric.pro` is also an Admin in the same org.
- For **cross-tenant / isolation** testing you need a second org: sign up a fresh alias like `roy.tolosa+otherorg@asymmetric.pro` (NOT invited to Acme). It auto-creates its own org, so you can confirm Acme's data is invisible from it.

3. Roles & the "tenant owner" rule

HazardOS is **multi-tenant** — every customer company is a separate organization, and nearly all data is scoped by organization via Postgres Row-Level Security. Permission order (highest to lowest):

platform_owner > platform_admin > tenant_owner > admin > estimator > technician > viewer

- **tenant_owner** — full access including billing.
- **admin** — manage team, configure settings, full operational access.
- **estimator** — create surveys, build estimates, send proposals.
- **technician** — see assigned jobs, log time/materials on completion.
- **viewer** — read-only.

Tenant Owner note (important for QA): the in-app *invite* flow deliberately does not offer Tenant Owner — only Admin/Estimator/Technician/Viewer. A second owner is created by *promotion*: an existing owner opens /settings/team and changes a member's role to Tenant Owner. The org supports multiple owners (there is no single-owner constraint); your +owner alias above is one such pre-provisioned owner. So test owner-level features by signing in as +owner, and test the promotion path from the owner account.

Multi-tenant isolation is the highest-severity thing to test. A user in Org A must never see or modify Org B's data. If you find a way to leak data across orgs, that's the top-priority bug in the platform.

4. Signing in & browser session tips

- Sign in at /login. Sign up at /signup (a fresh, un-invited alias auto-creates a new org with you as tenant_owner).
- Use one normal window for one role and an incognito/private window for another, so you can be two roles at once.
- Use a third browser entirely (e.g. Firefox next to Chrome) for the second-organization isolation test.
- **Mobile:** install the PWA on your phone (Safari iOS / Chrome Android) to test the field survey honestly. This matters most for Roy.

5. Who tests what

QA is split between two testers. In the tracker, filter the **Tester** column to your name to get your worklist.

Tester	Owns	Persona
Sophie Hope	Auth + multi-tenant isolation, CRM, estimates, jobs (scheduling + variance), invoices (creation + delivery), pipeline + analytics, commissions, settings + integrations, billing	The internal / back-office user — what the remediation company sees

Roy Tolosa	Mobile site survey + offline sync, proposal customer portal, customer feedback portal, SMS opt-in / TCPA, mobile + offline cross-cutting, invoices received as a customer	The field user + the customer experience
------------	---	--

Both own: the smoke pass (whoever hits a deploy first), security-conscious testing (sanity-check each other's findings, especially cross-tenant), accessibility on your own surfaces, and exploratory.

Cross-boundary workflows (pair up — schedule ~30 min each, both signed in)

1. **Estimate → proposal → signature → job.** Sophie sends a proposal from an estimate; Roy opens the customer portal link in incognito, reviews and e-signs; Sophie confirms the signed proposal auto-creates a job in her scheduler with correct status.
2. **Job completion → invoice → customer receipt.** Roy completes a job on mobile (or Sophie marks one complete); Sophie reviews variance, approves, triggers the invoice; Roy receives it by email AND SMS per preference and confirms the customer-facing view is clean (no internal IDs).
3. **Customer feedback loop.** Sophie triggers the feedback survey on a paid job; Roy submits NPS + testimonial; Sophie confirms the score lands in analytics.
4. **Multi-tenant isolation.** Both sign up fresh +otherorg aliases as separate orgs, capture a few resource URLs, then try to open each other's URLs while in their own org — every attempt must be blocked.

6. Using the tracker (HazardOS QA Tracker spreadsheet)

1. Every test step is one row: **Pass | Pass Name | Section | Test ID | Test Step | Status | Notes | Bug Link | Tester | Date.**
2. Filter the **Tester** column to your name; sort by Pass number and work through them in order.
3. Set each row's **Status** from the dropdown: **Not Started / Pass / Fail / Blocked / N/A** (Fail turns red, Pass green, Blocked amber automatically).
4. Add a note, and a bug link (or pasted bug write-up) in the **Bug Link** column when something fails. Fill in the **Date**.
5. Use **one shared sheet**, not two — Mark watches the same sheet. When a bug crosses your boundary, tag Mark in a comment so triage knows it's a handoff.

The file you were sent is an .xlsx. Drag it into Google Drive (or File → Import) and it opens as a Google Sheet with the dropdowns and color-coding intact.

7. The test passes (run in order)

Do **Smoke** first to confirm a build isn't catastrophically broken, then the deeper passes. Times are rough estimates.

No.	Pass	Est.
1	Smoke	60 min
2	Authentication & onboarding	60 min
3	CRM	90 min
4	Site survey (mobile-first)	120 min
5	Estimates	60 min
6	Proposals & customer portal	60 min
7	Jobs & job completion	90 min
8	Invoices & payments	60 min
9	Pipeline & analytics	45 min
10	Commissions	30 min
11	Customer feedback portal	30 min
12	SMS opt-in & communications	45 min
13	Settings & integrations	60 min
14	Cross-cutting tests	120 min
15	Security-conscious testing	60 min
16	Exploratory / "try to break it"	open-ended

8. Critical workflows that MUST work

These are the platform's value proposition. If any breaks, it's a critical bug.

1. **Estimate-to-job:** estimator surveys a property on their phone (survives going offline mid-walkthrough) → survey converts to an estimate → sent as a proposal → customer opens the portal link, reviews the PDF, e-signs → a job is auto-created → scheduler sees it.
2. **Job completion:** crew opens the assigned job on mobile → logs start time, materials, photos → submits → office reviews variance vs estimate → invoice auto-generated → sent by email + SMS → payment recorded → job marked paid.
3. **Customer feedback:** paid + complete job → feedback link sent → customer submits NPS + testimonial on the public portal → score shows in analytics.
4. **Multi-tenant isolation:** a user in Org A sees ONLY Org A's data; even direct URL access to an Org B resource from an Org A session must 403 or 404.

9. What is NOT a bug (don't log these)

- The platform admin panel says "Unauthorized" — correct, you're not platform_admin.
- The mobile survey shows "Offline — your changes will sync when you're back online" — by design.
- Customer links (proposal portal, feedback survey, SMS opt-in) work WITHOUT a login — those are intentionally public via token URLs.
- Photos taking a moment to appear after upload — background processing queue.
- API endpoints returning 401 without an API key — correct.
- A new signup landing on Starter tier with limited features — upgrade path is expected.
- A brand-new account's dashboard being empty — there's no seed data.

Known issues — already tracked, please don't re-log

- Security: 23 npm audit vulns; Stripe webhook secret empty-string fallback in dev; timing-unsafe cron secret compare; /api/errors/report unauthenticated; missing rate limiting on public API v1; debug pages open to any authed user; raw Supabase errors sometimes surfaced to clients.
- Auth: new signups get role "owner" in some paths instead of the canonical "tenant_owner" — doesn't crash, but it's wrong.
- Performance: useMultiTenantAuth fires 2–3 requests per mount with no cache; notification bell polls 2 endpoints every 30s; select('*') on wide tables in ~27 places.
- Test coverage sits at ~42%; auth, Stripe, survey store, and photo queue are at 0%.

Fair to log: anything not on those lists, anything visibly broken in user-facing flows, performance issues you personally hit (slow loads, flicker, double-loads), and anything that contradicts what this guide says should happen.

10. Bug report format

Keep bug reports short and declarative. Four things, in this order — Mark wants to read each in 10 seconds and know exactly what to fix:

1. **What you're seeing** (and where, exactly)
2. **What you expected**
3. **How the result is different**
4. **Desired end-state** (what should happen instead)

Example:

When you start a mobile survey and go offline mid-walkthrough, the draft saves to IndexedDB but the photo upload queue silently fails on reconnect — uploads don't retry.

Expected: queued photos retry up to 3 times when the network returns.

Difference: queue stays full, no retry attempts visible in the Network tab.

Desired: automatic retry on reconnect, with a retry count in the sync indicator.

Add these only when not obvious: account/tier (if it differs by role), browser/device (if you can't reproduce elsewhere), screenshot/video (visual issues), "reproducible?" (if intermittent), severity (critical = data loss / cross-tenant leak / can't sign in / can't pay; high = core workflow broken; medium = works but broken UX; low = cosmetic), and console errors (paste if it looks like a crash). Log each bug as a row in the tracker's Bug Link column, or as a GitHub issue if you

have access.

11. When to ask Mark

Only if you hit one of these: you need a test fixture (e.g. an org pre-loaded with 100 customers and 50 jobs for performance testing); you think you've found a cross-tenant data leak and want to confirm before reporting; or you need platform_admin access to test cross-tenant admin tooling.